

Oracle Lineage AI (Prototipo MVP)	1
1. Descripción de la Necesidad y Resultado Esperado	1
1.1. Necesidad a Resolver	1
1.2. Resultado Esperado	1
1.3. Sustento Técnico y Referencias Oficiales	1
A. Extracción de Lógica de Negocio (Código Fuente)	1
B. Validación de Dependencias (Nivel Objeto)	2
C. Justificación del Parser Propio (Nivel Campo)	2
2. Alcance del Proyecto	2
2.1. In Scope	2
2.2. Out of Scope	3
3. Entregables	3
4. Requerimientos	3
4.1. Funcionales	4
4.2. Técnicos y de Infraestructura	4
4.3. Herramientas y Licencias	4
5. Metodología	4
Etapas del Proyecto	4
6. Plan de Trabajo	4
7. Cronograma y Tiempos	5
8. Recursos Necesarios	5
8.1. Equipo Humano (The Squad)	5
8.2. Presupuesto Estimado (Operativo Cloud & AI)	6
9. Supuestos	6
10. Restricciones	6
11. Riesgos y Mitigación	6
12. Criterios de Aceptación	7
13. Plan de Pruebas	7
14. Plan de Transición y Puesta en Producción	7
15. Plan de Mantenimiento	8

Oracle Lineage AI (Prototipo MVP)

1. Descripción de la Necesidad y Resultado Esperado

1.1. Necesidad a Resolver

La organización enfrenta una barrera crítica para la modernización de su plataforma de datos: la falta de visibilidad sobre la lógica de negocio incrustada en bases de datos Oracle heredadas. Los arquitectos carecen de herramientas para trazar el impacto de los cambios, ya que el linaje de datos está oculto en código PL/SQL complejo y dinámico ("Cajas Negras"). Esto incrementa exponencialmente el riesgo de rotura de procesos y reportes durante las migraciones, obligando a revisiones manuales costosas y propensas a error.

1.2. Resultado Esperado

Desarrollar y desplegar "Oracle Lineage AI", un prototipo de aplicación web contenerizada que actúa como un Asistente Inteligente de Linaje.

El sistema deberá:

1. **Ingestar y analizar** esquemas de bases de datos Oracle de forma automática.
2. **Generar un grafo de linaje a nivel de campo**, resolviendo tanto dependencias estáticas (vía parser determinístico) como dinámicas (vía inferencia por IA Generativa).
3. **Visualizar** los resultados en una interfaz web interactiva que permita a los arquitectos navegar desde un reporte final hasta la tabla origen, identificando claramente qué relaciones son certeras y cuáles son inferidas.

1.3. Sustento Técnico y Referencias Oficiales

Nuestro enfoque de extracción se basa estrictamente en la arquitectura de **Vistas del Diccionario de Datos Estáticos** de Oracle. No utilizamos tablas propietarias ni agentes intrusivos. A continuación, se detalla el uso específico de cada vista oficial y su referencia documental:

A. Extracción de Lógica de Negocio (Código Fuente)

- **ALL_SOURCE:**
- **Uso en el Proyecto:** Fuente primaria para extraer el código PL/SQL de **Procedimientos, Funciones, Paquetes (Packages) y Triggers**. El agente concatena el campo TEXT ordenado por LINE para reconstruir la lógica completa que será analizada por el Parser y la IA.
- **Documentación:** [Oracle Database Reference: ALL_SOURCE](#)
- **ALL_VIEWS:**
- **Uso en el Proyecto:** Fuente para extraer la definición SQL (SELECT) de las Vistas lógicas. Se extrae el campo TEXT (o TEXT_VC en versiones recientes) para parsear las proyecciones y mapear el linaje columna a columna.
- **Documentación:** [Oracle Database Reference: ALL_VIEWS](#)
- **ALL_MVIEWS:**
- **Uso en el Proyecto:** Fuente para Vistas Materializadas. Se extrae el campo QUERY para entender la consulta base que genera la tabla física, permitiendo trazar el linaje hasta las tablas origen subyacentes.
- **Documentación:** [Oracle Database Reference: ALL_MVIEWS](#)

B. Validación de Dependencias (Nivel Objeto)

- **ALL_DEPENDENCIES:**
- **Uso en el Proyecto:** Se utiliza como "Ground Truth" (Verdad Terrestre) para validar el grafo generado. Si nuestro parser detecta una relación, esta debe existir también en ALL_DEPENDENCIES (dependencia a nivel objeto). Nos permite certificar que no estamos inventando relaciones inexistentes para el motor.
- **Documentación:** [Oracle Database Reference: ALL_DEPENDENCIES](#)

C. Justificación del Parser Propio (Nivel Campo)

Oracle documenta que su rastreo de dependencias nativo es "Coarse-Grained" (Grano Grueso/Nivel Objeto). Para lograr el objetivo de este proyecto (Trazabilidad "Fine-Grained" a nivel de Columna), es mandatorio realizar el análisis sintáctico (parsing) del código extraído de las vistas anteriores, práctica estándar en herramientas de Data Governance.

2. Alcance del Proyecto

El alcance se define bajo el principio de "**Inteligencia Aumentada**": automatizar lo masivo y asistir en lo complejo.

2.1. In Scope

- **Backend Híbrido:**
- Implementación de un Parser Determinístico (basado en sqlglot y Python) para el análisis de tablas, vistas y PL/SQL estándar.
- Implementación de un **Módulo de Fallback con IA (LLM)**: Integración con API de Modelos de Lenguaje (ej. GPT-4o vía Azure OpenAI) para analizar fragmentos de código donde el parser falle (SQL Dinámico, lógica ofuscada).
- **Frontend Interactivo:**
- Desarrollo de una SPA (Single Page Application) en **ReactJS** utilizando la librería de componentes **MUI (Material UI)**.
- Implementación de visualización de grafos navegables mediante **Apache ECharts**.
- Funcionalidad de búsqueda de objetos y resaltado de rutas (path highlighting).
- **Infraestructura:**
- Contenerización de la solución (Docker).
- Despliegue en **Azure Container Apps** (o servicio equivalente de contenedores en nube).
- **Entregables de Datos:**
- Inventario de objetos y reporte de complejidad/cobertura (Qué porcentaje fue resuelto por Parser vs IA).

2.2. Out of Scope

- **Ejecución de Código:** El agente no ejecutará sentencias SQL en la base de datos (solo lectura de metadatos y código).
- **Edición de Modelos:** La herramienta es de visualización y consulta, no de modelado de datos (tipo ERwin) ni de gobierno (tipo Collibra).
- **Alta Disponibilidad (HA):** Al ser un prototipo, no se contemplan configuraciones de Disaster Recovery o redundancia geográfica.
- **Linaje de Transformaciones Externas:** No se analizará código fuera de la base de datos (ej. Jobs de ETL en Informatica o DataStage).

3. Entregables

ID	Entregable	Descripción
E01	Código Fuente (Repositorio)	Repositorio Git con el Backend (Python/FastAPI), Frontend (React) y Dockerfiles. Incluye tests unitarios asistidos por Copilot.
E02	Aplicación Desplegada	URL accesible del prototipo ejecutándose en Azure, conectada a la base de datos de prueba.
E03	Diccionario de Datos Visual	Interfaz donde cada objeto tiene su ficha técnica y su micro-grafo de linaje renderizado.
E04	Reporte de "Cajas Negras"	Listado de procedimientos que ni el Parser ni la IA pudieron resolver con alta confianza.
E05	Documentación Técnica	Diagramas de arquitectura, guía de despliegue y documentación de la API REST.

4. Requerimientos

4.1. Funcionales

- El sistema debe permitir al usuario ingresar los parámetros de conexión de una BD Oracle.
- El sistema debe identificar automáticamente si un bloque de código es SQL Estático o Dinámico.
- El sistema debe visualizar el linaje con distinción visual (ej. colores de arista) entre relaciones "Determinísticas" (Parser) y "Probabilísticas" (IA).
- El usuario debe poder hacer clic en un nodo (campo/tabla) y ver sus dependencias inmediatas (upstream/downstream).

4.2. Técnicos y de Infraestructura

- **Nube:** Azure (Subscription activa para Container Apps y Azure OpenAI Service).

- **Contenedores:** Docker para empaquetado y distribución.
- **Backend:** Python 3.10+, Framework FastAPI o Flask.
- **Frontend:** React 18+, Material UI 5+, Apache ECharts.
- **Base de Datos Origen:** Oracle 11g R2, 12c, 19c o superior.

4.3. Herramientas y Licencias

- **IA de Desarrollo:** GitHub Copilot (para aceleración de codificación).
- **IA de Análisis:** API Key para LLM (OpenAI / Azure OpenAI).
- **Control de Versiones:** Git / GitHub o Azure DevOps.

5. Metodología

Se utilizará una metodología **Iterativa Incremental** con ciclos de 2 semanas (Sprints), apoyada fuertemente en **AI-Driven Development** (uso de Copilot para boilerplate y refactorización).

Etapas del Proyecto

1. **Fundamentos (Sprints 1-2):** Setup de infraestructura Azure, Boilerplate de React y Python, Conectividad básica a Oracle.
2. **Core Parser (Sprints 3-4):** Adaptación del motor sqlglot y lógica de grafos (networkx). Validación con SQL estático.
3. **Inteligencia Híbrida (Sprints 5-6):** Implementación del "LLM Handler", ingeniería de prompts para SQL dinámico e integración de resultados.
4. **Experiencia de Usuario (Sprints 7-8):** Construcción de la UI en React, integración de ECharts y API REST.
5. **Despliegue y Cierre (Sprint 9):** Deploy en Azure, UAT (User Acceptance Testing) y documentación final.

6. Plan de Trabajo

Fase	Tarea Clave	Responsable	Dependencia
1. Inicio	Configuración de Entorno Azure y Repositorios.	Fullstack Dev	N/A
	Desarrollo de OracleConnector y extracción de DDL.	AI Backend Eng	Accesos BD
2. Backend	Implementación de Parser Estático (tablas/vistas).	AI Backend Eng	Extracción DDL
	Desarrollo de LLMAalyzer para SQL Dinámico.	AI Backend Eng	API Key LLM

	Exposición de API REST (Endpoints de Grafo).	AI Backend Eng	Parser + LLM
3. Frontend	Maquetado de UI con MUI (Búsqueda, Paneles).	Fullstack Dev	API REST Mock
	Integración de ECharts con datos reales.	Fullstack Dev	API REST Real
4. DevOps	Creación de Dockerfiles y Pipeline de Build.	Fullstack Dev	Código Estable
	Despliegue en Azure Container Apps.	AI Backend Eng	Suscripción Azure

7. Cronograma y Tiempos

Duración Total Estimada: 10 Semanas (2.5 Meses).

- **Semana 1:** Kick-off y Setup de Entorno Cloud.
- **Semana 4:** Hito 1 - Backend capaz de parsear Vistas y Tablas estáticas.
- **Semana 7:** Hito 2 - Integración del módulo de IA y API funcional.
- **Semana 9:** Hito 3 - Frontend terminado y conectado.
- **Semana 10:** Hito Final - Despliegue productivo y Demo.

8. Recursos Necesarios

8.1. Equipo Humano (The Squad)

- **1 AI Backend Engineer (Líder Técnico):** Experto en Python, SQL Parsers e integración de LLMs. (Dedicación Alta).
- **1 Fullstack Developer:** Especialista en React, TypeScript y Visualización de Datos. (Dedicación Alta).
- **1 Oracle SME (Consultor):** Experto en PL/SQL para validación de lógica y casos de prueba. (Dedicación Parcial/Bajo Demanda).

8.2. Presupuesto Estimado (Operativo Cloud & AI)

- **Azure Container Apps:** ~\$50 - \$100 USD/mes (Entorno de desarrollo y pruebas).
- **API LLM (Tokens):** ~\$100 - \$300 USD (Dependiendo del volumen de PL/SQL dinámico a analizar).
- **GitHub Copilot:** ~\$38 USD/mes (2 licencias).

9. Supuestos

- Se cuenta con acceso de red (VPN o IP pública permitida) desde el entorno de Azure hacia la base de datos Oracle objetivo.
- La organización proveerá las credenciales de base de datos con los permisos necesarios (SELECT ANY DICTIONARY).
- El uso de servicios de IA en la nube (Azure OpenAI) está permitido por las políticas de seguridad de la empresa (los datos de metadata viajarán a la API).

10. Restricciones

- **Seguridad de Datos:** No se puede enviar *data real* (filas de clientes) al LLM, solo *metadata* (código SQL, nombres de tablas). El código debe sanitizarse antes del envío.
- **Presupuesto de Tokens:** Existe un límite mensual de consumo de API de IA que no debe superarse sin aprobación.
- **Ventanas de Mantenimiento:** El escaneo de la base de datos debe ser bajo demanda o programado fuera de horario pico para no afectar el rendimiento de la BD productiva.

11. Riesgos y Mitigación

Riesgo	Probabilidad	Impacto	Estrategia de Mitigación
Alucinaciones del LLM: La IA inventa una relación de linaje incorrecta.	Media	Alto	Diferenciación Visual: Marcar claramente en la UI las relaciones inferidas. Implementar validación cruzada básica (verificar si la tabla "alucinada" existe en el diccionario).
Bloqueo de Firewall: Azure no puede ver la BD Oracle On-Premise.	Alta	Alto	Utilizar una VNet Integration o desplegar un agente híbrido (Hybrid Worker) dentro de la red on-premise que empuje los datos a la nube.
Complejidad de Visualización: El	Alta	Medio	Implementar "Micro-Grafos"

grafo es ilegible por exceso de nodos.			(ver solo N niveles de profundidad) y funciones de colapso de nodos en ECharts.
--	--	--	---

12. Criterios de Aceptación

1. **Conectividad:** La aplicación se conecta exitosamente a la instancia Oracle y extrae el catálogo.
2. **Cobertura Híbrida:** El sistema parsea correctamente el 100% de las dependencias explícitas (ALL_DEPENDENCIES) y ofrece una sugerencia lógica para al menos el 70% de los bloques de SQL dinámico identificados.
3. **Usabilidad:** La interfaz web carga el grafo de una tabla promedio en menos de 3 segundos y permite la navegación interactiva (zoom, pan, click).
4. **Despliegue:** La solución corre en contenedores en Azure sin errores de inicio.

13. Plan de Pruebas

- **Pruebas Unitarias (Backend):** Tests automáticos para validar que el parser de sqlglot extrae correctamente inputs/outputs de strings SQL conocidos.
- **Pruebas de "Ground Truth" (IA):** Se tomarán 10 procedimientos complejos ya conocidos y validados manualmente por el SME. Se comparará el resultado del Agente vs la realidad para calibrar los Prompts.
- **Pruebas de UI/UX:** Navegación de flujo completo por parte de un usuario final (Arquitecto) para validar que la visualización es comprensible.

14. Plan de Transición y Puesta en Producción

- **Entrega de Artefactos:** Transferencia de imágenes Docker al Container Registry corporativo.
- **Capacitación:** Sesión de demo grabada y documentación de "Cómo interpretar el Grafo (Certeza vs Inferencia)".
- **Configuración:** Entrega de scripts de Terraform o Bicep para recrear la infraestructura en otros entornos.

15. Plan de Mantenimiento

- **Soporte Nivel 3 (Desarrollo):** El equipo del proyecto dará soporte de *bugfix* durante 2 semanas posteriores al despliegue.
- **Monitoreo:** Configuración de alertas en Azure Monitor para vigilar el consumo de CPU de los contenedores y el gasto de la API de OpenAI.
- **Evolución:** Las mejoras al Prompt del sistema (para mejorar la precisión de la IA) serán gestionadas como cambios de configuración en el repositorio, no requieren recompilar todo el código.